Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-8 — Hola TSOFT

1. CONTEXTO
Se requiere una página web estática que muestre saludos personalizados por país (oficinas TSOFT), con menú desplegable de banderas, diseño responsivo y cambio dinámico de saludo al seleccionar un país. El requerimiento menciona 'solo HTML5 y CSS' en los criterios de aceptación, pero también exige interactividad (menú desplegable funcional, cambio de saludo dinámico), lo que hace IMPRESCINDIBLE el uso de JavaScript. No hay backend, no hay base de datos, no hay autenticación. El entorno de ejecución es LOCAL (protocolo file:///) ya que no se menciona ningún servidor.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single File Application (SFA) — HTML estático con JS embebido o en archivo plano único.

Dado que el entorno es LOCAL (file:///), se aplica la restricción crítica del skill:
- 'L PROHIBIDO: `<script type='module'>`, `import/export` ES6 nativos, múltiples archivos JS con dependencias entre sí.
- ' PERMITIDO: Un único archivo `js/main.js` con JavaScript vanilla plano (IIFE o script clásico), referenciado con `<script src='js/main.js'></script>` sin atributo `type='module'`.

La estructura de carpetas solicitada en architecture_requirements (/css/, /js/, /images/) se respeta, pero la carpeta /images/ NO contendrá archivos binarios. Las banderas se renderizarán usando emojis Unicode nativos del sistema operativo (Ø<ÝæØ<Ý÷, Ø<ÝòØ<Ýý, Ø<ÝúØ<Ýø, etc.), eliminando cualquier dependencia de archivos .png compatibilidad total en entorno local sin servidor.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS / BORING TECHNOLOGY
- **¿Por qué NO microservicios?** No hay equipos independientes, no hay APIs, no hay datos que persistan. Un microservicio para mostrar un saludo estático sería ingeniería de ciencia ficción.
- **¿Por qué NO React/Vue/Angular?** YAGNI absoluto. Un framework JS para 6-8 saludos estáticos introduce: node_modules (200MB+), proceso de build, configuración de bundler, y una curva de aprendizaje innecesaria. Vanilla JS resuelve el 100% del requerimiento.
- **¿Por qué NO SASS/LESS?** El requerimiento menciona SASS como opción futura ('a medida que la aplicación crezca'). Hoy, con un único archivo CSS, agregar un preprocesador requiere Node.js, npm y un proceso de compilación. YAGNI: se usa CSS puro con variables nativas (--color-primary, etc.) que ofrecen el 80% del beneficio de SASS sin ninguna dependencia.
- **¿Por qué NO múltiples archivos JS?** Entorno local (file:///) hace que los módulos ES6 fallen por política CORS del navegador. Un solo archivo main.js es la única solución robusta.
- **¿Por qué NO base de datos?** Los saludos son datos estáticos conocidos (máximo 10 países). Un objeto JSON hardcodeado en main.js es suficiente, mantenible y no requiere infraestructura.
- **¿Por qué NO CDN externo para fuentes o iconos?** En entorno local sin garantía de internet, las dependencias externas son un riesgo. Se usan Google Fonts con fallback seguro y emojis nativos.

4. PAÍSES TSOFT (Datos de negocio asumidos)
Basado en el contexto de TSOFT como empresa latinoamericana de tecnología, se incluyen: Ø<ÝæØ<Ý÷ Argentina, Ø<ÝòØ Colombia, Ø<ÝèØ<Ýñ Chile, Ø<ÝõØ<Ýê Perú, Ø<ÝúØ<Ýþ Uruguay, Ø<ÝçØ<Ý÷ Brasil, Ø<ÝúØ<Ýø USA. Cada saludo usa '¡Che, bienvenido al palo!', México: '¡Qué onda wey, bienvenido!', Colombia: '¡Quiubo parce, bienvenido!', etc.).

5. SEGURIDAD (CSP y XSS)
- **XSS**: No hay inputs de usuario. Los datos de saludos son estáticos en el código. Se usa `textContent` en lugar de `innerHTML` para cualquier inserción dinámica de texto, eliminando vectores XSS.
- **CSP**: Se implementa via `<meta http-equiv='Content-Security-Policy'>` en el HTML (válido para entorno local). Política: `default-src 'self'; style-src 'self' 'unsafe-inline' fonts.googleapis.com; font-src fonts.gstatic.com; script-src 'self'; img-src 'self' data:;`.
- **HTTPS**: Recomendación documentada en el README. Para producción, desplegar en servidor con TLS. No aplicable en entorno local.
- **No cookies / no localStorage**: La aplicación no almacena ningún dato en el cliente.

6. PALETA DE COLORES TSOFT
Se asume paleta corporativa estándar para empresas tech latinoamericanas: Azul primario (#0A2463), Azul acento (#3E92CC), Naranja energía (#FF6B35), Blanco (#FFFFFF), Gris claro (#F5F7FA). Estas variables se definen en :root del CSS.

7. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1**: Si en el futuro se requiere agregar 50+ países con saludos editables por CMS, el objeto JSON hardcodeado deberá migrarse a una API. Mitigación: el objeto de datos está aislado en una constante `TSOFT_GREETINGS` al inicio de main.js, facilitando la extracción futura.
- **Riesgo 2**: Los emojis de banderas pueden no renderizarse en Windows 10 (limitación del SO, no del código). Mitigación: se agrega el nombre del país como texto visible siempre.
- **Riesgo 3**: Sin SASS, los estilos futuros pueden volverse difíciles de mantener. Mitigación: uso de CSS Custom Properties (variables) y comentarios de sección estructurados.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- js/main.js | Estado: APROBADO | Iteraciones: 2
- css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 44,877
Tokens de salida (output): 14,660
Total tokens: 59,537
Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~14.9 horas-hombre ahorradas
"H squad de 4 personas × 1.9 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“2,727 input | !‘1,924 output | 1 llamadas | ~1.8h ahorradas
MICROPLANIFICACION-DISEÑO: !“12,116 input | !‘6,000 output | 4 llamadas | ~5.4h ahorradas
FRONTEND-DESARROLLO: !“13,492 input | !‘6,016 output | 5 llamadas | ~4.3h ahorradas
QA-VALIDACION: !“16,542 input | !‘720 output | 5 llamadas | ~3.4h ahorradas